Information Sciences 667 (2024) 120429

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

SecureTLM: Private inference for transformer-based large model with MPC

Yuntian Chen, Xianjia Meng *, Zhiying Shi, Zhiyuan Ning, Jingzhi Lin

School of Information Science & Technology, Northwest University, 710062, Xi'an, China

ARTICLE INFO

Keywords:
Secure multiparty computation
Transformer
Private inference
Hybrid secret sharing

ABSTRACT

Transformer-based Large Models (TLM), such as generative pre-trained models (GPT), have become increasingly popular for practical applications through Deep Learning as a Service (DLaaS). They have been extensively used in natural language processing and computer vision. However, concerns regarding potential private data leakage arise with this type of inference service. While some private inference techniques can protect privacy, they often introduce high latency and approximate replacements in the design protocols, resulting in changes to the model structure and decreased accuracy. In this research, we present SecureTLM, a private inference method based on secure multi-party computation (MPC) that does not require modifications to the underlying model structure. SecureTLM offers protocols for crucial computations in TLM, such as Multiplication, Softmax, GeLU, and LayerNorm, without altering the model structure. Experimental results demonstrate that SecureTLM ensures data privacy, maintains correctness, and achieves efficiency in private inference tasks.

1. Introduction

The emergence of the transformer [1] has garnered attention among researchers due to its remarkable processing capabilities in various Natural Language Processing (NLP) [2] tasks such as machine translation [3], question answering [4], and text generation [5]. Moreover, transformer variants, like the vision transformer [6], have sparked significant interest in the realm of computer vision, where they have been utilized for tasks such as image generation and classification. Additionally, a multitude of other transformer variations have been investigated [7], showcasing impressive performance across a plethora of tasks.

The burgeoning availability of large-scale data [8] has significantly expanded the potential of the neural networks. As a result, the pre-trained transformer-based large models (TLM) have merged, they can be fine-tuned for downstream classification or generation tasks. These pre-trained TLM, such as Generative Pre-trained Transformers (GPT) [9] and Bidirectional Encoder Representations from Transformers (BERT) [10], have already demonstrated tremendous potential in diverse scenarios, such as search engines [11], recommendation systems [12], intelligent interactions [4] and creative writing [13].

However, the significant scale of the TLM, such as GPT-3 with 175 billion parameters, BERT-base with 110 million parameters, and GPT-2 with 1.5 billion parameters [14], presents challenges for users with limited computational resources when deploying these models. As a result, TLM are commonly deployed on cloud servers rather than local computing resources, following the

* Corresponding author.
E-mail addresses: chenyt@stumail.nwu.edu.cn (Y. Chen), xianjiam@nwu.edu.cn (X. Meng), shizhiying@stumail.nwu.edu.cn (Z. Shi), ningzhiyuan@stumail.nwu.edu.cn (Z. Ning), 2020117234@stumail.nwu.edu.cn (J. Lin).

https://doi.org/10.1016/j.ins.2024.120429
Received 26 December 2023; Received in revised form 2 March 2024; Accepted 3 March 2024

Available online 7 March 2024
0020-0255/© 2024 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Deep Learning as a Service (DLaaS) [15] paradigm. In this setup, users send inputs to the server for processing and receive the results in return. However, this service paradigm raises privacy concerns [16]. For instance, during the autonomous creative process using AI-generated (Artificial Intelligence) content, the inputs and outputs of the model may contain the thoughts and ideas of the creators, potentially leading to potential infringement of their interests. Similarly, in certain medical dialogue scenarios, the model's inputs and outputs may expose users' private information. Moreover, there is a risk that service providers may inadvertently disclose the parameters of their models. These privacy issues usually arise during the inference phase of the models. The lack of privacy protections during inference can hinder privacy-conscious users from sharing their data with service providers, potentially hampering data-driven model evolution for the service providers.

As an effective privacy protection framework for deep learning, private inference [17,18] aims to safeguard the server's model weights from users while ensuring the server remains unaware of users' private input information. In a private inference framework, the user submits an encrypted input to the server, and the model provider transmits the encrypted weights to the server. Then the server performs computations to obtain the encrypted inference results that can only be decrypted by the user. This method ensures that the service provider cannot access any private input or output information, and conversely, the user does not receive any information other than the private output. In order to achieve private inference, some cryptographic techniques, such as Secure Multi-Party Computation (MPC) [19] and Homomorphic Encryption (HE) [20], are utilized and have yielded certain effects.

In recent years, there has been significant development in neural network private inference for traditional CNN (Convolution Neural Network) and RNN (Recurrent Neural Network) models. CryptFlow [21] and CryptGPU [22] have demonstrated the feasibility of privacy-preserving inference in CNN-based visual neural networks. [23] and [24] introduced two fast privacy inference techniques for CNN-based neural networks based on Fully Homomorphic Encryption (FHE). [25] focuses on secure matrix multiplication for graph convolutional neural networks. [26] have proposed the PrivSeq2Seq framework, a privacy inference framework for sequence models based on Long Short Term Memory network (LSTM), which has been successfully applied to protect user privacy data in NLP tasks. These works have achieved effective results in traditional visual and textual privacy protection tasks.

Nevertheless, directly extending it to support the private inference for TLM is not feasible. The TLM exhibits a significant expansion in the number of parameters, involving larger-scale linear operations and more sophisticated nonlinear computations. Specifically, TLM consists of multiple transformer blocks, the multiplication scale is much higher than the traditional convolutional networks. And traditional convolutional neural networks, such as LetNet, commonly employ Rectified Linear Unit (ReLU) and batch normalization as activation functions, while TLM often utilizes activation functions such as the Gaussian Error Linear Unit (GeLU), extensively employ Softmax functions and LayerNorm techniques. Moreover, these complex nonlinear functions are not computation-friendly for encryption primitives, which makes inference less efficient [27].

Existing private inference techniques for TLM [28,29], which employ hybrid cryptographic technology, aiming to achieve private inference for TLM by substituting complex operators with privacy-computing friendly approximations (e.g., replacing GeLU with ReLU). However, these substitutions often lead to a decrease in model accuracy, necessitating retraining or fine-tuning to maintain precision. [30] introduced THE-X, which utilizes homomorphic encryption technology but fits the Softmax function with a three-layer fully connected network, while removing the pooling layer that contains the tanh operation. MPCFormer [31] realizes TLM privacy reasoning based on secret sharing but still adopts the approximate substitution method. CipherGPT [32] and BOLT [33] implement two-party secure inference of GPT, using RLWE-based (Ring Learning with error) HE technology for linear computations, and piecewise approximation substitution for non-linear computations. Consequently, this approach still results in a decrease in model accuracy, and the authors employ a similar strategy by conducting retraining to preserve model precision.

Although these methods solve the problem of TLM privacy protection to a certain extent, there are still some limitations:

- **Flawed approximate substitution.** Using friendly approximations such as polynomial and ReLU functions to replace expensive functions like GeLU and Softmax to reduce cost. However, simple substitution can result in a significant decrease in Transformer model accuracy, which may require extra model retraining or fine-tuning.
- **High inference cost.** Due to the multi-head attention mechanism, TLM models often require a large number of matrix multiplications. The existing cryptographic techniques for private matrix multiplication are based on HE and OT. However, both of these approaches have drawbacks, the HE technology, relying on public key cryptography, involves the generation of public and private keys and carries out computations on ciphertext, leading to an increased computational burden. And OT-based private matrix multiplication has a shorter computation time, but it incurs additional communication overheads during protocol execution.

It can be seen that some schemes pursue efficiency through crude replacement of complex functions, which improves the computational efficiency to a certain extent but compromises the accuracy of the model. Therefore, designing a private inference framework for pre-trained TLM that avoids retraining or fine-tuning, ensures efficiency, and maintains accuracy equivalent to plain-text inference poses a significant challenge.

1.1. Our contributions

This study introduces a flexible MPC-based private inference framework for TLM, known as SecureTLM, which encompasses the user side and the MPC server side. The parties in the MPC server can be flexibly applied to any number according to specific scenarios. In this framework, we designate the user as the "Dealer" [34], tasked with generating the Beaver's triplet [35] required for the multiplication protocol during the preprocessing stage, and transmitting them to the MPC server through trusted transmission.

This configuration can make the MPC server complete the multiplication protocol with just one round interaction, and avoid the use of OT and HE to generate the cross terms in the Beaver's triple. As a result, this approach significantly enhances the efficiency of the multiplication process.

For the rough substitutions, the primary reason is that cryptographic techniques are not friendly to nonlinear computations. Such replacements would result in a loss of model accuracy. To address this, we design accuracy lossless protocols for nonlinear activation functions utilized in TLM such as Softmax and GeLU which are based on HSS (Hybrid Secret Sharing) techniques (both additive shared and multiplicative shared [36]). To the best of our knowledge, we are the first to use the HSS technique in TLM. Based on these protocols, SecureTLM does not require changing any components in the TLM model. Thus obviating the necessity for additional model retraining or fine-tuning when using the proposed private protocols.

Furthermore, SecureTLM comprises a compilation of diverse underlying protocols, necessitating the assurance of the security of each sub-protocol. Therefore, in the subsequent parts of the research, we provide security proofs for the sub-protocols. Building upon this groundwork, we implemented and integrated each sub-protocol to construct the SecureTLM system. Through a series of experiments and evaluations under various settings, we have validated the accuracy and efficiency of the protocols.

Our contributions are summarized as follows:

- We propose SecureTLM, an efficient cryptographic framework for the private inference TLM in the "Dealer"-based preprocessing setting.
- We design protocols for multiplication, Softmax, GeLU, and LayerNorm, which are used in the TLM.
- The security proof of the protocols we designed is provided in the later sections of this work. We still also show the accuracy guarantee and the improvement in efficiency of SecureTLM through experiments.

1.2. Organization of the paper

The remaining sections of this paper are organized as follows: 2 provides a concise review of the relevant background material. 3 presents the system architecture, while 4 describes the building blocks and the private inference process. 5 offers a formal proof of the protocols, followed by 6 evaluating the performance of the protocol. Finally, we conclude this work in 7.

2. Preliminaries

2.1. Notations

In this work, we use $\mathbf{A_i}$ to indicate the row of the matrix $\mathbf{A}$, and a_i to represent the element of a vector $\mathbf{a}$. We denote by $\{\mathrm{n}\}$ the set $\{1, ..., N\}$ for $\{n\} \in \mathbb{N}$. Here, $\mathbb{N}$ is the set of parties in MPC, hence, the party can be denoted as $P_{\{i\}}$. The additive and multiplicative secret sharing of $x \in \mathbb{R}$ are denoted as $[x]_{\{i\}}$ and $\langle x \rangle_{\{i\}}$ respectively, where x is a number in rational field $\mathbb{R}$. Given $\mathbf{y} \in \mathbb{R}^{d_m}$, d_m refers to the vectors of m length, and for $\mathbf{M} \in \mathbb{R}^{d_m \times d_k}$, $d_m \times d_k$ is a matrix with size $m \times k$.

2.2. Transformer-based large models (TLM)

Transformers are widely used in NLP tasks. The architecture of the transformer comprises the encoder and decoder. The encoder generates hidden states in a sequence based on the input sequence provided. Subsequently, the decoder takes these hidden states produced by the encoder and generates the output sequence. Real-world models often stack multiple encoder and decoder blocks. Furthermore, TLM usually adopts the decoder-only mode, such as GPT. As Fig. 1 shows. In the following section, we discuss the essential components of a single transformer block.

2.2.1. Attention layer

An attention function can be defined as a mapping between a query matrix $\mathbf{Q}$ and a set of key-value pairs matrices $(\mathbf{K}, \mathbf{V})$, resulting in an output. In this mapping, both the query and the keys, values, and output are represented as vectors. Specifically, the output is determined by computing a weighted sum. Given three matrices $\mathbf{Q}$ (query), $\mathbf{K}$ (key), $\mathbf{V}$ (value) in the latent represent. This attention layer can be formalized as eq. (1):

$$Attention(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = Softmax(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}})\mathbf{V} \tag{1}$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{d_s \times d_k}$. The Softmax function applies to each row vector of a matrix. For a vector $\mathbf{x} \in \mathbb{R}^s$. The Softmax on $\mathbf{x}$ returns a vector $\mathbf{y} \in \mathbb{R}^s$ can be described as eq. (2):

$$Softmax(x_i) = \frac{e^{x_i}}{\sum_{j=0}^{k-1} e^{x_j}} \tag{2}$$

The multi-head attention can be further extended by employing the aforementioned mechanism. Considering h different attention mechanisms with weight matrices $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V$ for $i = 1, ..., h$, we have

$$head_i = Attention(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V) \tag{3}$$

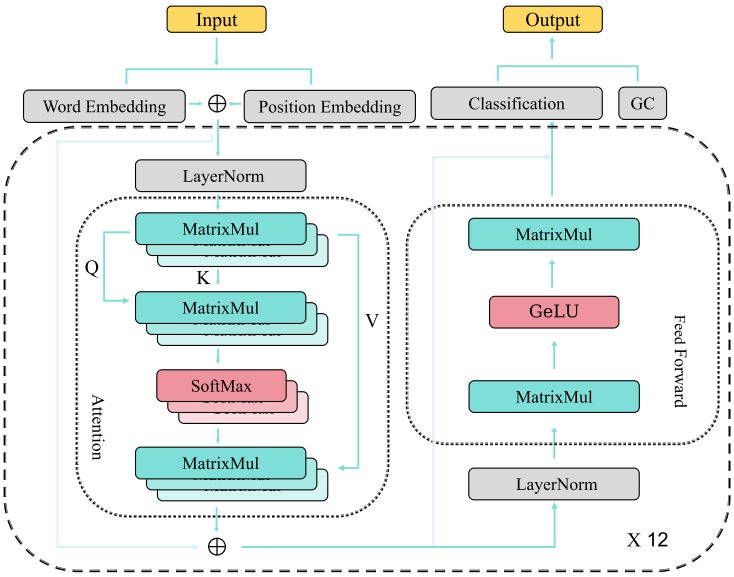

Fig. 1. The architecture of GPT.

where $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V \in \mathbb{R}^{d_k \times d_m}$, and $head_i \in \mathbb{R}^{d_s \times d_m}$. Then, the multi-head attention can be formalized as (4):

$$MultiHead(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = Concat(head_1, ..., head_h)\mathbf{W}^O \tag{4}$$

where $\mathbf{W}^O \in \mathbb{R}^{hd_k \times d_m}$, and MultiHead$(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{d_S \times d_m}$

2.2.2. Feed-forward layer (FFN)

The FFN consists of two fully connected layers and is separated by a GeLU activation. Let $\mathbf{X} \in \mathbb{R}^{d_s \times d_m}$ be an input, where d_s is the sequence length. FFN can be represented as eq. (5):

$$FFN(\mathbf{X}) = GeLU(\mathbf{X}\mathbf{W}^{F1} + \mathbf{B}^{F1})\mathbf{W}^{F2} + \mathbf{B}^{F2} \tag{5}$$

where $\mathbf{W}^{F1} \in \mathbb{R}^{d_m \times d_f}$, $\mathbf{W}^{F2} \in \mathbb{R}^{d_m \times d_f}$ are the weight matrices and $\mathbf{B}^{F1} \in \mathbb{R}^{d_f}$ $\mathbf{B}^{F2} \in \mathbb{R}^{d_m}$ are the bias vectors for the two layers of FNN. GeLU is the Gaussian Error Linear Unit activation function, which returns a vector $\mathbf{y} \in \mathbb{R}^k$ for the input vector $\mathbf{x} \in \mathbb{R}^k$, as the formula (6):

$$GeLU(x_i) = \frac{x_i}{2} \cdot (1 + tanh(\sqrt{\frac{2}{\pi}} \cdot (x_i + 0.044715 \cdot x_i^3))) \tag{6}$$

where tanh is the hyperbolic tangent function, which can be expressed by the formula (7):

$$tanh(\mu_i) = \frac{e^{2\mu_i} - 1}{e^{2\mu_i} + 1} \tag{7}$$

2.2.3. LayerNorm layer

Let $\mathbf{x} = (x_1, x_2, ..., x_k)$ be a vector representation of an input of size d_k to normalization layers. LayerNorm re-centers and re-scales input $\mathbf{x} \in \mathbb{R}^{\mathbf{d_k}}$ as (8):

$$LayerNorm(x_i) = \mathbf{G} \cdot \frac{x_i - \mu}{\sigma} + \mathbf{B} \tag{8}$$

where $\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$, and $\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$. μ and σ are the mean and standard deviation of $\mathbf{x}$. Bias $\mathbf{B}$ and gain $\mathbf{G}$ are parameters with the same dimension N.

2.3. Cryptographic primitives

2.3.1. Secure multi-party computation (MPC)

MPC [19] is a cryptographic technology that allows multiple participants $P_{\{1\}}, P_{\{2\}}, ..., P_{\{N\}}$ to jointly compute a specific function while ensuring the confidentiality of the individual private data. More specifically, for a function ξ with N-inputs and N-outputs, the objective is to perform secure computation $\xi([x]_{\{1\}}, [x]_{\{2\}}, ..., [x]_{\{N\}}) = ([y]_{\{1\}}, [y]_{\{2\}}, ..., [y]_{\{N\}})$ while ensuring the following properties:

- Participant $P_{\{i\}}$ can obtain the correct result $y_{\{i\}}$.
- Participant $P_{\{i\}}$ will not obtain any additional information beyond what can be derived from $x_{\{i\}}$ and $y_{\{i\}}$.

This technology provides a promising solution for preserving the privacy of data during the inference process of deep learning. Which incorporates complex cryptographic primitives such as Secret Sharing (SS), Oblivious Transfer (OT), and Garbled Circuit (GC) [21].

2.3.2. Secret sharing (SS)

Let $x \in \mathbb{R}$ represent a secret value, the data owner distributes the secret data x among various parties, with each party receiving a share of the secret. For each parity P_i where $i \in \mathbb{N}$, it obtains the additive share $[x]_{\{i\}}$ and the multiplicative share $\langle x \rangle_{\{i\}}$. To reconstruct the secret value, the data owner only needs to sum the additive shares and multiply the multiplicative share, the function can be denoted as $\mathcal{F}_{Rec}$. To simplify notation, we use + and × to denote operations between additive and multiplicative shared values, respectively. Thus, for an additive secret value of u, we have:

$$\mathcal{F}_{Rec}(u) = \sum_{i=\{1\}}^{\{N\}} [u]_{\{i\}} = [u]_{\{1\}} + [u]_{\{2\}} + \ldots + [u]_{\{N\}} \tag{9}$$

and for a multiplicative secret value of v, we have:

$$\mathcal{F}_{Rec}(v) = \prod_{i=\{1\}}^{\{N\}} \langle v \rangle_{\{i\}} = \langle v \rangle_{\{1\}} \times \langle v \rangle_{\{2\}} \times \ldots \times \langle v \rangle_{\{N\}} \tag{10}$$

If α is public constant and $\langle v \rangle_{\{i\}}$, $[u]_{\{i\}}$ are secret shared values, then, $\alpha \times [u]_{\{i\}} = [\alpha \times u]_{\{i\}}$, and $c^{[v]_{\{i\}}} = \langle c^v \rangle_{\{i\}}$. For these linear operations, each party can compute their respective shares locally based on the secrets they hold.

2.3.3. Multiplication ($\mathcal{F}_{mul}$)

Let u and v denote two secret values, the multiplication of the secret values can be calculated: $[w]_{\{i\}} + \ldots + [w]_{\{N\}} \underleftarrow{\mathcal{F}_{mul}} ([u]_{\{i\}} + \ldots + [u]_{\{N\}}) \times ([v]_{\{i\}} + \ldots + [u]_{\{N\}})$, namely $w = u \times v$.

2.3.4. Multiplicative shared transformation ($\mathcal{F}_{trans}$)

Let s denote a secret value, each parity obtains the multiplicative share $\langle s \rangle_{\{i\}}$ and the additive share $[t]_{\{i\}}$. Then the transformation process can be expressed as (11):

$$s = [t]_{\{1\}} + [t]_{\{2\}} + \ldots + [t]_{\{N\}} \underleftarrow{\mathcal{F}_{trans}} \langle s \rangle_{\{1\}} \times \langle s \rangle_{\{2\}} \times \ldots \times \langle s \rangle_{\{N\}} \tag{11}$$

that is $s = \sum_{i=\{1\}}^{\{N\}} [t]_{\{i\}} \underleftarrow{\mathcal{F}_{trans}} \prod_{i=\{1\}}^{\{N\}} \langle s \rangle_{\{i\}}$.

3. SecureTLM system architecture

3.1. System construct

To achieve privacy preservation in model inference, our designed framework SecureTLM considers a general private inference scenario on DLasS. The MPC server comprises multiple parties $P_{\{1\}}, \ldots, [P]_{\{N\}}$, each owning a pre-trained model with its parameters, while the user possesses private inputs. The objective is for the user to input private information to the server and obtain private output. The server collaborates with the inference task and returns the computation results to the user, ensuring that this process does not expose any information regarding the user's input and output. As Fig. 2 shows.

3.1.1. User

The user possesses private data and provides a portion of the foundational support required for secure protocols, including the generation of Beaver's triples ($z = u \times v$). Assuming a two-party scenario in MPC, the equation $[z]_{\{1\}} + [z]_{\{2\}} = ([u]_{\{1\}} + [u]_{\{2\}}) \times ([v]_{\{1\}} + [v]_{\{2\}})$ holds. Existing approaches involve the server generating Beaver's triples offline. However, the issue arises in the generation of cross terms when the server generates Beaver's triples. The traditional solution entails each MPC participant multiplying their possession of a secret value to achieve cross terms through either HE (e.g., Paillier) or OT. Nonetheless, using HE is unavoidably impacted by the computational efficiency constraints of public key cryptosystems. Similarly, employing OT necessitates additional encryption and authentication operations, thereby augmenting the communication burden. Therefore, it is realistic to regard the user (the data provider) as the "Dealer" in secure multi-party computation framework. The "Dealer" is responsible for locally pre-processing the cross-term and subsequently distributing it to each party through a trusted transmission.

3.1.2. MPC server

The MPC server comprises N parties, where each party owns the secret model parameters and the security protocols of the model. For the secret model parameters, similar to the User side, the model parameters are first divided into N parts, with each participant

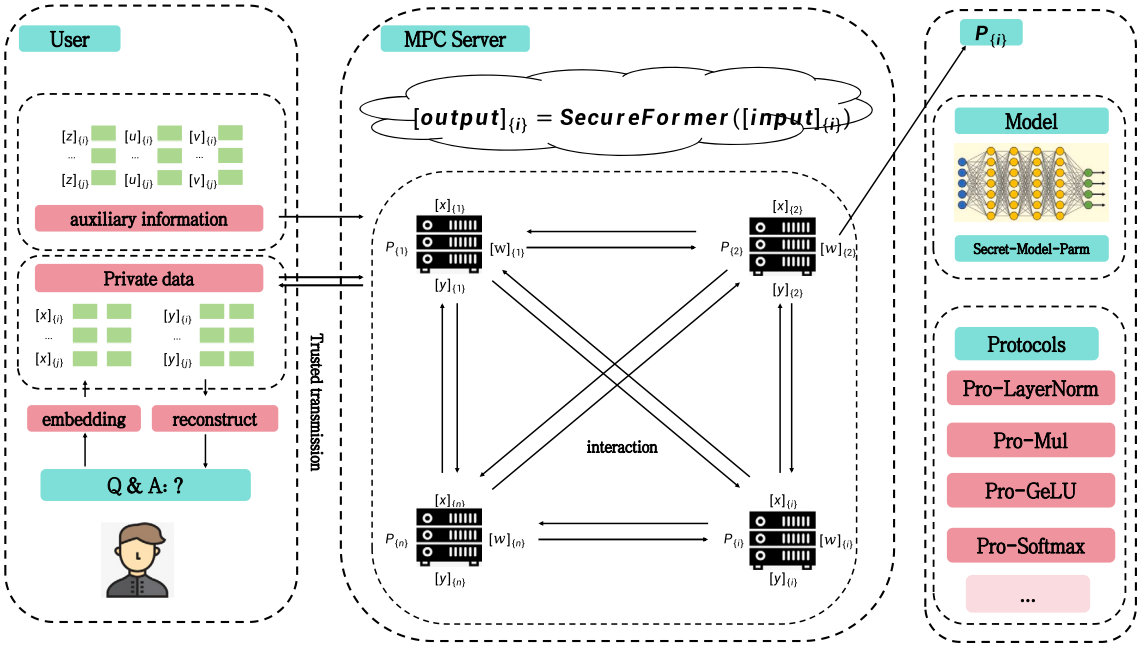

Fig. 2. The architecture of SecureTLM.

in the MPC server owning one part. This ensures that the MPC server side does not obtain the parameter information from the model provider. During the execution of the protocols, the computing party generates random numbers to participate in the calculation, ensuring that the User side cannot obtain the parameters information. Therefore, we refer to them as secret model parameters in the MPC server. When the user provides the pre-processed secret data, the MPC server executes the security protocols to complete the inference task, with each participant obtaining a secret value as the result. Subsequently, each computing party securely transmits its secret results to the user, then the user executes the reconstruction function to obtain the final inference results.

3.2. Thread model

This work considers a standard threat model known as honest-but-curious. The security is provably provided in simulation paradigm [37] against a static and honest-but-curious probabilistic polynomial-time (PPT) adversary $\mathcal{A}$. That is to say, the adversary $\mathcal{A}$ passively corrupts either (We consider the honest-but-curious adversary $\mathcal{A}$ that is able to corrupt at most $n-1$ servers, denote as $I \subset \mathbb{N}$) at the beginning of the protocol but honestly follows the protocol specification.

The simulation paradigm defines two distinct worlds: the real world and the ideal world. Assuming that there exists a simulator $\mathcal{S}$ can create an ideal world for a protocol $\mathcal{F}$ (equivalent to the ideal functionality $\mathcal{IF}$) where the server perform the protocol $\mathcal{F}$ according to the specification in the presence of $\mathcal{A}$. And an ideal world where the parties send their data to an ideal functionality that executes the evaluation faithfully. It is imperative to ensure that, for any adversary $\mathcal{A}$, the distribution in the real world is computationally indistinguishable from the ideal world:

$$\begin{aligned} &\{View_{\mathcal{A}}^{\mathcal{F}}(k, [x]_{\{1\}}, [x]_{\{2\}}, ..., [x]_{\{N\}}), output^{\mathcal{F}}(\mathbf{y})\} \overset{c}{\approx} \\ &\mathcal{S}(k, \{[x]_{\{i\}}, \mathcal{IF}([x]_{\{1\}})\}_{i \in I}, \mathcal{IF}([x]_{\{1\}}, [x]_{\{2\}}, ..., [x]_{\{N\}})) \end{aligned} \tag{12}$$

where k is the secure parameter. $View_{\mathcal{A}}^{\mathcal{F}}$ is the view of $P_{\{i\}}$ in the execution of $\mathcal{F}$ on $\mathbf{x}$, $output^{\mathcal{F}}$ is the output of all parties.

SecureTLM invokes several sub-protocols and we describe them using the hybrid model. This is similar to a real execution, except that sub-protocols are replaced by the invocations of the corresponding functionality instances. Formally, a protocol invoking a functionality $\mathcal{F}$ is called to be in $\mathcal{F}$-hybrid model [38].

3.3. Design goals

The SecureTLM framework aims to enable private inference for TLM that achieves privacy protection and protocols efficiency simultaneously, without compromising model accuracy. Specifically, this work demonstrates the following desirable goals:

- **Lossless inference accuracy**: Compared to inference tasks in plain-text settings, the designed protocols should not sacrifice inference accuracy.

- **Privacy protection**: In privacy-preserving inference tasks, the MPC server ensures the execution of computations according to the protocols. Each honest party only obtains its own shared results and is unable to access the shared values of other parties.
- **Efficient evaluation**: The privacy inference framework SecureTLM must fulfill the requirement of efficiency. For each participant, the computation and communication costs caused by the protocols should be as low as possible to accommodate diverse deployment scenarios.

4. Build blocks

Similar to traditional neural networks, TLM can also be categorized into linear layers and non-linear layers. In transformers, the linear layers are distinctive, incorporating attention mechanisms that entail extensive matrix multiplication operations. The non-linear layers consist of more sophisticated activation functions such as Softmax and GeLU. In this section, we describe the designed protocols for SecureTLM.

4.1. Secure linear protocols

4.1.1. Protocol for multiplication ($\mathcal{F}_{mul}$)

Like the implementation of the multiplication in the convolutional neural network private inference technique, Beaver's triples are also utilized to achieve our multiplication protocol. In previous studies, the parties should first generate the Beaver's triple $z = u \times v$ in the offline phase. Assuming the existence of N participants, the equation $[z]_{\{1\}} + [z]_{\{2\}} + ... + [z]_{\{N\}} = ([u]_{\{1\}} + [u]_{\{2\}} + ... + [u]_{\{N\}}) \times ([v]_{\{1\}} + [v]_{\{2\}} + ... + [v]_{\{N\}})$ holds.

The generation of Beaver's triple computation faces challenges when calculating cross-terms, which require interactive computation by each party using HE or OT technology. With the inspiration from [26], the secret share values of each party and the necessary cross-term calculations for Beaver's triple generation are obtained through "Dealer" preprocessing, wherein SecureTLM designates the user as the "Dealer". Then triples are pre-generated and transmitted to all participants of the MPC server trusted channels.

In the context, assuming the given input x and y, w is the product of x and y, denoted as $w = x \times y$. Each party locally computes $[a]_{\{i\}} = [x]_{\{i\}} - [u]_{\{i\}}$, $[b]_{\{i\}} = [y]_{\{i\}} - [b]_{\{i\}}$, and reconstruct a, b and then open to each other. Finally, the parity $P_{\{i\}}$ holds that $[w]_{\{i\}} = a \times b + b \times [u]_{\{i\}} + a \times [v]_{\{i\}} + [z]_{\{i\}}$, other parities $P_{\{o\}}, o \in \mathbb{N} \setminus \{i\}$ hold $[w]_{\{o\}} = b \times [u]_{\{o\}} + a \times [v]_{\{o\}} + [z]_{\{o\}}$. Each party can obtain the additive secret shared value of the multiplication result without the need for interaction by performing offline calculations. Algorithm 1 describes N-PC protocol for private multiplication, where parties hold shared values of x and y. The multiplication protocol $\mathcal{F}_{mul}$) relies on Beaver's triples. In the initial phase, parties also hold shared values of z, u and v. And $\mathcal{F}_{mul}$) outputs the shares of the result w.

Algorithm 1 Protocol $\mathcal{F}_{mul}(P_{\{0\}}, P_{\{1\}}, ..., P_{\{N\}})$.

Input $P_{\{i\}}$ holds $[x]_{\{i\}}, [y]_{\{i\}}, [u]_{\{i\}}, [v]_{\{i\}}$ and $[z]_{\{i\}}$.
Output $P_{\{i\}}$ holds $[w]_{\{i\}}$. $P_{\{o\}}$ holds $[w]_{\{o\}}$.

Procedure
1. $P_{\{i\}}$ locally computes $[a]_{\{i\}} = [x]_{\{i\}} - [u]_{\{i\}}$ and $[b]_{\{i\}} = [y]_{\{i\}} - [v]_{\{i\}}$
2. Reconstruct $a = \sum_{i=\{1\}}^{\{N\}} [a]_{\{i\}}$, $b = \sum_{i=\{1\}}^{\{N\}} [b]_{\{i\}}$
3. $P_{\{i\}}$ locally computes $[w]_{\{i\}} = a \times b + b \times [u]_{\{i\}} + a \times [v]_{\{i\}} + [z]_{\{i\}}$
4. $P_{\{o\}}$ locally computes $[w]_{\{o\}} = b \times [u]_{\{o\}} + a \times [v]_{\{o\}} + [z]_{\{o\}}, o \in \mathbb{N} \setminus \{i\}$

Return $[w]_{\{i\}}, [w]_{\{o\}}$

4.1.2. Protocol for multiplication shared transformation ($\mathcal{F}_{trans}$)

To achieve lossless non-linear computation, the protocol utilizes a combination of additive and multiplicative mixed secret sharing. Therefore, a linear multiplicative secret-sharing conversion protocol is needed. Assuming the existence of parties $P_{\{1\}}, P_{\{2\}}, ..., P_{\{N\}}$, each $P_{\{i\}}$ possessing a multiplication shared value $\langle s \rangle_{\{i\}}$ of a secret value S. Upon executing the protocol $\mathcal{F}_{trans}$), each party obtains an addition secret shared value $[t]_{\{N\}}$ for the secret value S.

In previous research, they implement the transformation process using the partially homomorphic encryption algorithm Paillier [39]. The efficiency of homomorphic encryption computations affects this process. Another approach redivides the multiplication secret sharing value $\langle b \rangle_{\{i\}}$ into two additive secret sharing values $[c]_{\{i\}}$ and $[d]_{\{i\}}$, namely, $\langle b \rangle_{\{i\}} = [c]_{\{i\}} + [d]_{\{i\}}$. For a secret value B, it can be calculated as $B = \langle b \rangle_{\{1\}} \times \langle b \rangle_{\{2\}} \times ... \times \langle b \rangle_{\{N\}} = ([c]_{\{1\}} + [d]_{\{1\}}) \times ([c]_{\{2\}} + [d]_{\{2\}}) \times ... \times ([c]_{\{N\}} + [d]_{\{N\}})$ Through a reasonable interaction strategy, parties possess additive secret sharing values. However, this approach requires more communication, which implies certain limitations.

SecureTLM implements the protocol using auxiliary information [36]. Specifically, given a matrix $\mathbf{A} \in \mathbb{R}^{d_N \times d_N}$, let $\prod_{j=1}^{N} \alpha_{1j}$ represent the product of each row. Then the sum of the products of all rows can be stated that $\sum_{i=1}^{N} (\prod_{j=1}^{N} \alpha_{ij}) = 1$. Additionally given the multiplicative secret sharing $S = \prod_{j=1}^{N} \langle s \rangle_{\{i\}}$, each column of the matrix is multiplied by $\langle s \rangle_{\{i\}}$, resulting in new matrix $\mathbf{M} \in \mathbb{R}^{d_N \times d_N}$, where each element is $\langle s \rangle_{\{j\}} \times \alpha_{ij}$. In the end, each party possesses $[t]_{\{i\}} = \prod_{j=1}^{N} \alpha_{ij} \times \langle s \rangle_{\{j\}} = S \cdot \prod_{j=1}^{N} \alpha_{ij}$, and $\sum_{i=1}^{N} [t]_{\{i\}} = S \cdot \sum_{i=1}^{N} (\prod_{j=1}^{N} \alpha_{ij}) = S$. Algorithm 2 describes N-PC protocol for $\mathcal{F}_{trans}$.

Algorithm 2 Protocol $\mathcal{F}_{trans}(P_{\{0\}}, P_{\{1\}}, ..., P_{\{N\}})$.

Input $P_{\{i\}}$ holds $\langle s\rangle_{\{i\}}$.
Output $P_{\{i\}}$ holds $[t]_{\{i\}}$.

Procedure
1. **for** $i = 1$ **to** N
   for $j = 1$ **to** N
   $P_{\{i\}}$ locally computes $\langle s\rangle_{\{i\}} \times \alpha_{ij}$
2. $P_{\{i\}}$ locally computes $\prod_{j=1}^{N} \langle s\rangle_{\{j\}} \times \alpha_{ij} = [t]_{\{i\}}$

Return $[t]_{\{i\}}$

4.2. Secure non-linear protocols

This section primarily focuses on the implementation of protocols for nonlinear computation.

4.2.1. Protocol for attention ($\mathcal{F}_{att}$)

As described in 2.2.1, the attention layer can be divided into matrix multiplication ($\mathbf{M} = (\mathbf{Q} \odot \mathbf{K}) \times \frac{1}{\sqrt{d_k}}$), Softmax (the Softmax can be represented as ψ, such that $\mathbf{Y} = \psi(\mathbf{M})$), and matrix multiplication ($\mathbf{Y} \odot \mathbf{V}$). For matrix multiplication, we can describe the matrices using vectors and perform the protocol $\mathcal{F}_{mul}$ to obtain the results. The challenge aspect lies in implementing the protocol for $\varphi()$. In this study, a hybrid secret-sharing technique is employed to achieve $\mathcal{F}_{Softmax}$.

Specifically, given a vector $\mathbf{x} \in \mathbb{R}^{d_s}$, where x_m represents an element of $\mathbf{x}$, its additive secret sharing value is denoted as $[x_m]_{\{i\}}$. Initially, each party $P_{\{i\}}$ possesses a secret value $[x_m]_{\{i\}}$, and each $P_{\{i\}}$ locally computes $e^{[x_m]_{\{i\}}} \times \langle\gamma\rangle_{\{i\}}$, where γ is a random number and $\langle\gamma\rangle_{\{i\}}$ is the multiplicative secret sharing value of γ. According to the linear property of secret sharing described in 2.3.2, it can be concluded that $e^{[x_m]_{\{i\}}} \times \langle\gamma\rangle_{\{i\}} = \langle e^{x_m} \times \gamma\rangle_{\{i\}} = \langle\phi\rangle_{\{i\}}$. Then, each $P_{\{i\}}$ performs $\mathcal{F}_{trans}$, leading to $\{[\phi]_{\{1\}}, [\phi]_{\{2\}}, ..., [\phi]_{\{N\}}\} \xleftarrow{\mathcal{F}_{trans}} \{\langle\phi\rangle_{\{1\}}, \langle\phi\rangle_{\{2\}}, ..., \langle\phi\rangle_{\{N\}}\}$. Next, each $P_{\{i\}}$ performs $\mathcal{F}_{Rec}$ to get the result $\sum_{i=1}^{N} [\phi]_{\{i\}} = e^{x_m} \times \gamma = \varphi_m$. By executing the process in parallel, $P_{\{i\}}$ can be obtained $\varphi = \sum_{m=0}^{d_k} e^{x_m} \times \gamma$ efficiently. Finally, each party $P_{\{i\}}$ locally computes $\psi([x_m]_{\{i\}}) = \frac{[\phi]_{\{i\}}}{\varphi}$, and the result $\beta_m = \psi(x_m)$ for each party $P_{\{i\}}$ will be denoted as $[\beta_m]_{\{i\}}$, which is determined by the eq. (13):

$$\begin{aligned} \beta_m &= \sum_{i=\{1\}}^{\{N\}} \frac{[\phi]_{\{i\}}}{\varphi} = \frac{\sum_{i=\{1\}}^{\{N\}} [\phi]_{\{i\}}}{\varphi} \\ &= \frac{e^{x_m} \times \gamma}{\sum_{m=0}^{d_k} e^{x_m} \times \gamma} = \frac{e^{x_m}}{\sum_{m=0}^{d_k} e^{x_m}} \end{aligned} \tag{13}$$

The designed $\mathcal{F}_{Softmax}$ is shown in Algorithm 3.

Algorithm 3 Protocol $\mathcal{F}_{Softmax}(P_{\{0\}}, P_{\{1\}}, ..., P_{\{N\}})$.

Input $P_{\{i\}}$ holds $[x_m]_{\{i\}}$.
Output $P_{\{i\}}$ holds $[\beta_m]_{\{i\}}$.

Procedure
1. $P_{\{i\}}$ locally computes $\langle e^{x_m} \times \gamma\rangle_{\{i\}}$
2. $P_{\{i\}}$ performs $\mathcal{F}_{trans}$ to compute $\{[\phi]_{\{1\}}, [\phi]_{\{2\}}, ..., [\phi]_{\{N\}}\} \xleftarrow{\mathcal{F}_{trans}} \{\langle\phi\rangle_{\{1\}}, \langle\phi\rangle_{\{2\}}, ..., \langle\phi\rangle_{\{N\}}\}$
3. $P_{\{i\}}$ performs $\mathcal{F}_{Rec}$ to obtain $e^{x_m} \times \gamma \xleftarrow{\mathcal{F}_{Rec}} [\phi]_{\{i\}}$
4. Each $P_{\{i\}}$, through one interaction, computes $\varphi = \sum_{m=0}^{d_k} e^{x_m} \times \gamma$
4. $P_{\{i\}}$ locally computes $\psi([x_m]_{\{i\}}) = \frac{[\phi]_{\{i\}}}{\varphi}$

Return $[\beta_m]_{\{i\}}$

Then, the $\mathcal{F}_{att}$ can be obtained by invoking the $\mathcal{F}_{Softmax}$ and $\mathcal{F}_{mul}$, and shown in (14):

$$\begin{aligned} &[\mathbf{Z}]_{\{i\}} \xleftarrow{\mathcal{F}_{att}} ([\mathbf{Q}]_{\{i\}}, [\mathbf{K}]_{\{i\}}, [\mathbf{V}]_{\{i\}}) \\ &= \xleftarrow{\mathcal{F}_{mul}} ([\mathbf{Y}]_{\{i\}}, [\mathbf{V}]_{\{i\}}), \quad [\mathbf{Y}]_{\{i\}} \xleftarrow{\mathcal{F}_{sofmtax}} [\mathbf{M}]_{\{i\}}, \\ &[\mathbf{M}]_{\{i\}} = \{[\mathbf{V}]_i \xleftarrow{\mathcal{F}_{mul}} ([\mathbf{Q}]_{\{i\}}, [\mathbf{K}]_{\{i\}})\} \times \frac{1}{\sqrt{d_k}} \end{aligned} \tag{14}$$

4.2.2. Protocol for FFN ($\mathcal{F}_{ffn}$)

As described in 2.2.2, the FFN layer consists of two linear layers with a GeLU function inserted between them. Similar to Attention, the FFN can be divided into matrix multiplication, GeLU activation, and matrix multiplication. The matrix multiplication step can adopt similar processing methods as seen in Attention, with the key focus on the calculation of the GeLU function. Specifically, GeLU includes multiplication operations and tanh non-linear computation. Let x_m be an element from $\mathbf{x} \in \mathbb{R}$, for x_m^3 within GeLU, it can

be obtained by executing the multiplication protocol twice, while tanh (denoted as $\xi()$) can be implemented using a hybrid secret sharing technique.

Specifically, SecureTLM performs $\mathcal{F}_{mul}([x_m]_{\{i\}},[x_m]_{\{i\}})$ to produce $[x_m^2]_{\{i\}}$, and again perform $\mathcal{F}_{mul}([x_m]_{\{i\}},[x_m^2]_{\{i\}})$ to produce $[x_m^3]_{\{i\}}$. Addition and multiplication by scalar are performed subsequently before locally computing $[\alpha_m]_{\{i\}} = \sqrt{\frac{2}{\pi}}([x_m]_{\{i\}} + 0.0044715 \times [x_m^3]_{\{i\}})$. For *tanh*, assuming that the input $[\alpha_m]_{\{i\}}$, and a randomly multiplicative secret sharing $\langle\gamma\rangle_{\{i\}}$ is available for computation. To calculate $e^{2[\alpha_m]_{\{i\}}} \times \langle\gamma\rangle_{\{i\}}$, each party $P_{\{i\}}$ locally computes $e^{2[\alpha_m]_{\{i\}}} \times \langle\gamma\rangle_{\{i\}}$. According to the linear property of secret sharing, we have $e^{2[\alpha_m]_{\{i\}}} \times \langle\gamma\rangle_{\{i\}} = \langle e^{2\alpha_m} \times \gamma\rangle_{\{i\}} = \langle\theta_m\rangle_i$. Then, each $P_{\{i\}}$ performs $\mathcal{F}_{trans}$, namely $\{[\theta_m]_{\{1\}},[\theta_m]_{\{2\}},...,[\theta_m]_{\{N\}}\} \underline{\mathcal{F}_{trans}} \{\langle\theta_m\rangle_{\{1\}},\langle\theta_m\rangle_{\{2\}},\langle\theta_m\rangle_{\{N\}}\}$. And performs $\mathcal{F}_{trans}$ again to obtain $\{[\eta]_{\{1\}},[\eta]_{\{2\}},...,[\eta]_{\{N\}}\} \underline{\mathcal{F}_{trans}} \{\langle\gamma\rangle_{\{1\}},\langle\gamma\rangle_{\{2\}},...,\langle\gamma\rangle_{\{N\}}\}$. Next, $P_{\{i\}}$ locally calculates $[\omega_m]_{\{i\}} = [\theta_m]_{\{i\}} + [\eta]_{\{i\}}$, and the parties perform $\mathcal{F}_{Rec}$ to obtain $\omega_m \underline{\mathcal{F}_{Rec}} [\omega_m]_{\{i\}}$. Then, $P_{\{i\}}$ locally computes $[\beta_m]_{\{i\}} = \frac{[\theta_m]_{\{i\}}-[\eta]_{\{i\}}}{\omega_m}$. Finally, the share result of $\mathcal{F}_{gelu}$ is $[g_m]_{\{i\}} = \frac{[x_m]_{\{i\}}}{2} \times (1+[\beta_m]_{\{i\}})$, and the result $g_m = \xi(x_m)$ for each party $P_{\{i\}}$ will be denoted as $[g_m]_{\{i\}}$. For tanh, which is determined by the eq. (15):

$$\begin{aligned}\beta_m &= \sum_{i=\{1\}}^{\{N\}} \frac{[\theta_m]_{\{i\}} - [\eta]_{\{i\}}}{\omega_m} = \frac{\sum_{i=\{1\}}^{\{N\}}([\theta_m]_{\{i\}} - [\eta]_{\{i\}})}{\omega_m} \\ &= \frac{\theta_m - \eta}{\theta_m + \eta} = \frac{e^{2\alpha_m} \times \gamma - \gamma}{e^{2\alpha_m} \times \gamma + \gamma} = \frac{e^{2\alpha_m} - 1}{e^{2\alpha_m} + 1}\end{aligned} \tag{15}$$

The designed $\mathcal{F}_{gelu}$ is shown in Algorithm 4.

Algorithm 4 Protocol $\mathcal{F}_{gelu}(P_{\{0\}},P_{\{1\}},...,P_{\{N\}})$.

Input $P_{\{i\}}$ holds $[x_m]_{\{i\}}$.
Output $P_{\{i\}}$ holds $[g_m]_{\{i\}}$.

Procedure
1. parties run $\mathcal{F}_{mul}([x_m]_{\{i\}},[x_m]_{\{i\}})$ to compute $[x_m^2]_{\{i\}}$
2. parties run $\mathcal{F}_{mul}([x_m]_{\{i\}},[x_m^2]_{\{i\}})$ again to compute $[x_m^3]_{\{i\}}$
3. each party $P_{\{i\}}$ locally computes constant multiplication and simple addition to obtain $[\alpha_m]_{\{i\}}$
4. each party $P_{\{i\}}$ locally computes $e^{2[\alpha_m]_{\{i\}}} \times \langle\gamma\rangle_{\{i\}} = \langle e^{2\alpha_m} \times \gamma\rangle_{\{i\}}$
5. each party $P_{\{i\}}$ perform $\mathcal{F}_{trans}$ to obtain the sharing $\{[\eta]_{\{1\}},[\eta]_{\{2\}},...,[\eta]_{\{N\}}\} \underline{\mathcal{F}_{trans}} \{\langle\gamma\rangle_{\{1\}},\langle\gamma\rangle_{\{2\}},...,\langle\gamma\rangle_{\{N\}}\}$ and $\{[\theta_m]_{\{1\}},[\theta_m]_{\{2\}},...,[\theta_m]_{\{N\}}\} \underline{\mathcal{F}_{trans}} \{\langle\theta_m\rangle_{\{1\}},\langle\theta_m\rangle_{\{2\}},\langle\theta_m\rangle_{\{N\}}\}$
6. each party $P_{\{i\}}$ locally computes $[\omega_m]_{\{i\}} = [\theta_m]_{\{i\}} + \eta]_{\{i\}}$
7. parties run $\mathcal{F}_{Rec}$ to obtain ω_m
8. each party $P_{\{i\}}$ locally computes $[\beta_m]_{\{i\}} = \frac{[\theta_m]_{\{i\}}-[\eta]_{\{i\}}}{\omega}$
9. each party $P_{\{i\}}$ locally computes $[g_m]_{\{i\}} = \frac{[x_m]_{\{i\}}}{2} \times (1+[\beta_m]_{\{i\}})$

Return $[g_m]_{\{i\}}$

Similar to $\mathcal{F}_{att}$, $\mathcal{F}_{ffn}$ can be obtained by invoking $\mathcal{F}_{gelu}$ and $\mathcal{F}_{mul}$.

4.2.3. Protocol for LayerNorm ($\mathcal{F}_{layernorm}$)

In accordance with Section 2.2.3, consider a vector $\mathbf{x} \in \mathbb{R}^{d_k}$, where x_m represents an element of $\mathbf{x}$. The additive secret sharing value of x_m is denoted as $[x_m]_{\{i\}}$. To calculate the mean, firstly, each party $P_{\{i\}}$ calculates the secret sharing of mean $[\mu]_{\{i\}}$, which is equal to $[\mu]_{\{i\}} = \frac{\sum_{m=1}^{d_k}[x_m]_{\{i\}}}{d_k}$. Subsequently, $P_{\{i\}}$ calculate the secret sharing of variance $[\sigma^{-1/2}]_{\{i\}}$ using $\mathcal{F}_{mul}$ and $\mathcal{F}_{sqrt}$. Specifically, $P_{\{i\}}$ locally computes $[\alpha_m]_{\{i\}} = [x_m]_{\{i\}} - [\mu]_{\{i\}}$, next parties execute $[\delta_m]_{\{i\}} \underline{\mathcal{F}_{mul}} ([\alpha_m]_{\{i\}},[\alpha_m]_{\{i\}})$. After that, $P_{\{i\}}$ locally computes $[\sigma]_{\{i\}} = \frac{1}{d_k}\sum_{m=1}^{d_k}[\delta_m]_{\{i\}}$. Then we have $[\sigma^{-1/2}]_{\{i\}} \underline{\mathcal{F}_{sqrt}} [\sigma]_{\{i\}}$, where $\mathcal{F}_{sqrt}$ provided by [40]. Finally, parties parallel perform $\mathcal{F}_{mul}$ to obtain the secret sharing of the result $[v_m]_{\{i\}} = \mathcal{F}_{mul}([\alpha_m]_{\{i\}},[\sigma^{-1/2}]_{\{i\}})$ and $[\beta_m]_{\{i\}} = \mathcal{F}_{mul}([\mathbf{G}]_{\{i\}},[v_m]_{\{i\}}) + [\mathbf{B}]_{\{i\}}$.

The designed $\mathcal{F}_{layernorm}$ is shown in Algorithm 5.

4.3. The SecureTLM private inference workflow

In this section, we present the SecureTLM private inference overview based on the building blocks described in Section 4. Through interaction, each computing party executes the protocols and is unable to learn anything beyond the inputs and outputs. As shown in Fig. 3, we have only elucidated the private inference process of a single transformer decoder part. First, the user inputs a sequence of a specific length, represented as vector $\mathbf{x}$. In the preprocessing stage, the user divides the input vector x into N parts, each containing $[x]_{\{i\}}$, and securely transmits them to each participant of the MPC server. Subsequently, each computing party $P_{\{i\}}$ holds the secret input $[x]_{\{i\}}$ and secret parameters $[w]_{\{i\}}$, and by executing the protocols $\mathcal{F}_{layernorm}$, $\mathcal{F}_{mul}$, $\mathcal{F}_{softmax}$, $\mathcal{F}_{ffn}$, obtains the secret output $[y]_{\{i\}}$. Finally, each participant sends their obtained $[y]_{\{i\}}$ to the user, who can reconstruct the privacy-preserving inference result. The private inference of TLM can be readily obtained by expansion.

Algorithm 5 Protocol $\mathcal{F}_{layernorm}(P_{\{0\}}, P_{\{1\}}, ..., P_{\{N\}})$.

Input $P_{\{i\}}$ holds $[x_m]_{\{i\}}$, $[\mathbf{G}]_{\{i\}}$ and $[\mathbf{B}]_{\{i\}}$.
Output $P_{\{i\}}$ holds $[\beta_m]_{\{i\}}$.

Procedure

1. each party $P_{\{i\}}$ locally computes $[\mu]_{\{i\}} = \sum_{m=1}^{d_k} [x_m]_{\{i\}} / d_k$
2. each party $P_{\{i\}}$ locally computes $[\alpha_m]_{\{i\}} = [x_m]_{\{i\}} - [\mu]_{\{i\}}$
3. parties run $\mathcal{F}_{mul}([\alpha_m]_{\{i\}}, [\alpha_m]_{\{i\}})$ to compute:
   $[\delta_m]_{\{i\}} \ \underline{\mathcal{F}_{mul}} \ ([\alpha_m]_{\{i\}}, [\alpha_m]_{\{i\}})$
4. each party $P_{\{i\}}$ locally computes $[\sigma]_{\{i\}} = \frac{1}{d_k} \sum_{m=1}^{d_k} [\delta_m]_{\{i\}}$
5. parties run $\mathcal{F}_{sqrt}[\sigma]_{\{i\}}$ to compute $[\sigma^{-1/2}]_{\{i\}} \ \underline{\mathcal{F}_{sqrt}} \ [\sigma]_{\{i\}}$
6. parties run $\mathcal{F}_{mul}$ to compute $[v_m]_{\{i\}} = \mathcal{F}_{mul}([\alpha_m]_{\{i\}}, [\sigma^{-1/2}]_{\{i\}})$ and $[\beta_m]_{\{i\}} = \mathcal{F}_{mul}([\mathbf{G}]_{\{i\}}, [v_m]_{\{i\}}) + [\mathbf{B}]_{\{i\}}$

Return $[\beta_m]_{\{i\}}$

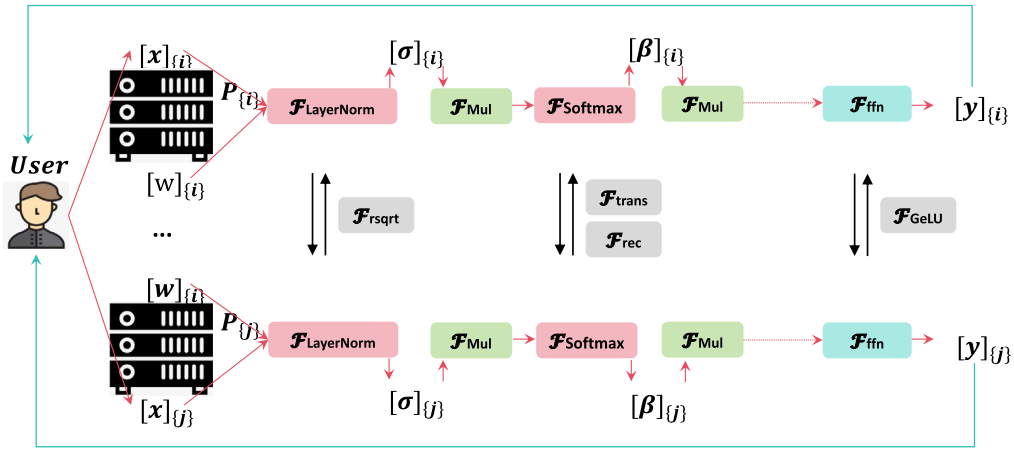

Fig. 3. The private inference workflow for Single transformer Decoder.

5. Security proof

In this section, we provide a theoretical roof on the security of SecureTLM. Similar to [40–42], this work is also based on the Universal Composability (UC)-framework [41]. As described in section 3.2, we aim to provide security guarantees for our proposed system in the presence of honest-but-curious adversary $\mathcal{A}$. Assuming that the honest-but-curious adversary $\mathcal{A}$ can control the computation parties in the cloud service, but has control over at most $n-1$ parties. In this work, we use $P_i, i \in \mathbb{I}$ denotes the honest parties, and $P_j, j \in \mathbb{J}$ denotes the corrupted parties, where $\mathbb{I} \cup \mathbb{J} = \mathbb{N}$ and $|\mathbb{J}| \leq n-1$. Here, security means that the adversary $\mathcal{A}$ only learns the inputs and output from the party that it has compromised, but nothing else beyond that [ref]. There are the following security definitions:

Definition 1. By stating that if there exists a polynomial-time simulator $\mathcal{S}$ capable of constructing a simulated world, where the view of an adversary $\mathcal{A}$ is computationally indistinguishable from the view in real-world, we can assert the security of the protocol. [37]

Lemma 1. *In the $\mathcal{F}$-hybrid model, a protocol is simulatable if all the protocols that invoke are simulatable. [37]*

Lemma 2. *The existence of $x \cdot r$ satisfying uniformly random if r is uniformly random. And $\mathcal{A}$ unknown r [43].*

Lemma 3. *The existence of $x \pm r$ satisfying uniformly random if r is uniformly random. And $\mathcal{A}$ unknown r [44].*

Lemma 4. *The multiplication protocol $\mathcal{F}_{mul}$ built by Beaver's triple is secure [35].*

Lemma 5. *The multiplication shared transformation protocol $\mathcal{F}_{trans}$ is secure [36].*

Lemma 6. *The multiplication shared transformation protocol $\mathcal{F}_{rsqrt}$ is secure [40].*

Our protocols are $\mathcal{F}$-hybrid model, based on the above lemmas, we first need to prove that the sub-protocols are secure. SecureTLM is secure when all sub-protocols are secure.

Theorem 1. *The protocol $\mathcal{F}_{Softmax}$ is secure against the honest-but-curious adversary in the case that $n-1$ parties are corrupted.*

Proof. The view of $\mathcal{F}_{Softmax}$ for corrupted party P_j is $view_j = \{[x_m]_{\{j\}}, \langle\gamma\rangle_{\{j\}}, \langle\phi\rangle_{\{j\}}, [\phi]_{\{j\}}, \varphi, \varphi_m, [\beta_m]_{\{j\}}\}$. It follows from Lemma 2 that $\langle\phi\rangle_{\{j\}} = e^{[x_m]_{\{j\}}} \times \langle\gamma\rangle_{\{j\}}$ is uniformly random. Because $\langle\gamma\rangle_{\{j\}}$ is uniformly random. According to Lemma 5, $\mathcal{F}_{trans}$ that has been proven secure in the honest-but-curious model, and $[\phi]_{\{j\}}$ are obtained through it. Thus $[\phi]_{\{j\}}$ is uniformly random. On the basis of Lemma 3, φ and φ_m are uniformly random. Then, $[\beta_m]_{\{j\}} = \frac{[\phi]_{\{i\}}}{\varphi}$ is still uniformly random. It turns out that the $view_j$ is easily simulatable by a probabilistic polynomial-time simulator $\mathcal{S}$ and is computationally indistinguishable from the real $view_j$ of $\mathcal{A}$. Thus, the protocol $\mathcal{F}_{Softmax}$ is secure against the honest-but-curious adversary. □

Theorem 2. *The protocol $\mathcal{F}_{att}$ is secure against the honest-but-curious adversary in the case that $n-1$ parties are corrupted.*

Proof. As shown in (14), protocol $\mathcal{F}_{att}$ is implemented by invoking $\mathcal{F}_{Softmax}$ and $\mathcal{F}_{mul}$. We have just proved that $\mathcal{F}_{Softmax}$ can be against the honest-but-curious adversary. Drawing from Lemma 4, it can be deduced that $\mathcal{F}_{mul}$ is also secure in the honest-but-curious model, so $\mathcal{F}_{att}$ is simulatable by $\mathcal{A}$. In other words, $\mathcal{F}_{att}$ is secure in the honest-but-curious model. □

Theorem 3. *The protocol $\mathcal{F}_{gelu}$ is secure against the honest-but-curious adversary in the case that $n-1$ parties are corrupted.*

Proof. The view of $\mathcal{F}_{gelu}$ for corrupted party P_j is $view_j = \{[x_m]_{\{j\}}, [x_m^2]_{\{j\}}, [x_m^3]_{\{j\}}, [\alpha_m]_{\{j\}}, [\theta_m]_{\{j\}}, [\omega_m]_{\{j\}}, [\beta_m]_{\{j\}}, [g_m]_{\{j\}}, \langle\gamma\rangle_{\{j\}}, \langle\theta_m\rangle_{\{j\}}, [\eta]_{\{j\}}, \omega_m\}$. $[x_m^2]_{\{j\}}$ and $[x_m^3]_{\{j\}}$ are obtained through $\mathcal{F}_{mul}$, and by Lemma 4, they are uniformly random. According to Lemma 2 and Lemma 3, $[\alpha_m]_{\{j\}} = \sqrt{\frac{2}{\pi}}([x_m]_{\{j\}} + 0.0044715 \times [x_m^3]_{\{j\}})$ is also uniformly random. Due to $\langle\gamma\rangle_{\{j\}}$ is uniformly random, $\langle\theta_m\rangle_{\{j\}} = \langle e^{2\alpha_m} \times \gamma\rangle_{\{j\}}$ still uniformly random on the basis of Lemma 2. According to Lemma 5, $[\theta_m]_{\{j\}}$ and $[\eta]_{\{j\}}$ are out from $\mathcal{F}_{trans}$, hence they are still uniformly random. Both ω_m, $[\omega_m]_{\{j\}} = [\theta_m]_{\{j\}} + [\eta]_{\{j\}}$, $[\beta_m]_{\{j\}} = \frac{[\theta_m]_{\{j\}} - [\eta]_{\{j\}}}{\omega_m}$ and $[g_m]_{\{j\}} = \frac{[x_m]_{\{j\}}}{2} \times (1 + [\beta_m]_{\{j\}})$ are from constant multiplication and simple addition, by the Lemma 2 and Lemma 3, it follows that both of ω_m, $[\omega_m]_{\{j\}}$, $[\beta_m]_{\{j\}}$ and $[g_m]_{\{j\}}$ are uniformly random. Consequently, there exists a probabilistic polynomial time simulator $\mathcal{S}$, which can simulate a view that is computation indistinguishable from the real $view_j$. Thus, $\mathcal{F}_{gelu}$ can against the honest-but-curious adversary. □

Theorem 4. *The protocol $\mathcal{F}_{ffn}$ is secure against the honest-but-curious adversary in the case that $n-1$ parties are corrupted.*

Proof. Similar to $\mathcal{F}_{att}$, $\mathcal{F}_{ffn}$ is obtained by invoking $\mathcal{F}_{mul}$ and $\mathcal{F}_{gelu}$. From Lemma 1, Lemma 4 and Theorem 3, we know that $\mathcal{F}_{ffn}$ is still secure in the honest-but-curious model. □

Theorem 5. *The protocol $\mathcal{F}_{layerNorm}$ is secure against the honest-but-curious adversary in the case that $n-1$ parties are corrupted.*

Proof. The view of protocol $\mathcal{F}_{layernorm}$ for corrupted $P_{\{j\}}$ is $view_j = \{[x_m]_{\{j\}}, [\mu]_{\{j\}}, [\alpha_m]_{\{j\}}, [\delta_m]_{\{j\}}\}$ and $\{[\sigma]_{\{j\}}, [\sigma^{-1/2}]_{\{j\}}, [v_m]_{\{j\}}, [\beta_m]_{\{j\}}, [\mathbf{G}]_{\{j\}}, [\mathbf{B}]_{\{j\}}\}$. As the intermediate value $[\delta_m]_{\{j\}}$ is outputs from $\mathcal{F}_{mul}$, it is uniformly random on the basis of Lemma 4. Likewise, $[\alpha_m]_{\{j\}} = [x_m]_{\{j\}} - [\mu]_{\{j\}}$, $[\mu]_{\{j\}} = \sum_{m=1}^{d_k} [x_m]_{\{j\}}/d_k$ and $[\sigma]_{\{j\}} = \sum_{m=1}^{d_k} [\delta_m]_{\{j\}}/d_k$ are still uniformly random according the Lemma 2 and Lemma 3. $[\sigma^{-1/2}]_{\{j\}}$ is given by $\mathcal{F}_{rsqrt}$, in accordance with Lemma 6, $[\sigma^{-1/2}]_{\{j\}}$ can against the honest-but-curious adversary. According Lemma 4, $[v_m]_{\{i\}} = \mathcal{F}_{mul}([\alpha_m]_{\{i\}}, [\sigma^{-1/2}]_{\{i\}})$ and the intermediate result $\mathcal{F}_{mul}([\mathbf{G}]_{\{j\}}, [v_m]_{\{i\}})$ is uniformly random. Furthermore, based on Lemma 3, the sum of the intermediate result and $[\mathbf{B}]_{\{j\}}$ is $[\beta_m]_{\{i\}}$, which is also uniformly random. Consequently, the view simulated by the simulator $\mathcal{S}$ is computationally indistinguishable from the real view of adversary $\mathcal{A}$. Thus, $\mathcal{F}_{layerNorm}$ is secure in the honest-but-curious model. □

Theorem 6. *The system SecureTLM can guarantee security in the honest-but-curious model in the case that at most $n-1$ parties are corrupted.*

Proof. In summary, we have proved the security of all sub-protocols that SecureTLM needs to invoke, and by Lemma 1, it can easily prove SecureTLM is secure in the honest-but-curious model. □

6. Experimental analysis

We discussed the protocols that are on the infinite field $\mathbb{R}$, nevertheless, real-world computers can only handle decimal numbers with limited precision and can only operate on numbers within a certain range. Therefore, for additive shares, fixed-point representation is suitable. Similar to the setting of [43], we implement our protocols with a finite field of fixed-point representation, and multiplicative shares often convert to additive shares locally, which can also be encoded in fixed-point representation to guarantee precision. Furthermore, we have already demonstrated the security of SecureTLM as mentioned above. In this section, we aim to validate the accuracy and performance of SecureTLM through experiments. We implemented the protocols used in SecureTLM with Python and C++ and ran our experiments on Intel(R) Xeon(R) Gold 6238R CPU @ 2.20 GHz with 377 GB RAM with a single thread in LAN settings.

Table 1
Accuracy of $\mathcal{F}_{Softmax}$ protocol.

Protocol	input	secret shares		secret result	error
		share A	share B		
$\mathcal{F}_{Softmax}$	−0.0614	−0.0689	0.0075	0.4912	$8.5 * 10^{-17}$
	−0.0265	0.0721	−0.0986	0.5087	$7.4 * 10^{-17}$
	0.0295	0.0915	−0.0620	0.5219	$4.4 * 10^{-17}$
	−0.0580	0.0182	−0.0762	0.4781	$6.3 * 10^{-17}$

Table 2
Accuracy of $\mathcal{F}_{gelu}$ protocol.

Protocol	input	secret shares		secret result	error
		share A	share B		
$\mathcal{F}_{gelu}$	−0.1388	0.1955	−0.3343	−0.0617	$0.1 * 10^{-17}$
	−0.4226	0.1678	−0.5904	−0.1426	$0.2 * 10^{-17}$
	1.4362	0.4741	0.9620	1.3067	$0.3 * 10^{-17}$
	−0.8081	−0.3850	−0.4231	−0.1740	$0.3 * 10^{-17}$

Table 3
Accuracy of $\mathcal{F}_{invsquare}$ protocol.

Protocol	input	secret shares		secret result	error
		share A	share B		
$\mathcal{F}_{invsquare}$	0.5103	0.4000	0.1103	1.4000	$2.27 * 10^{-13}$
	0.5621	0.4252	0.1369	1.3338	$4.09 * 10^{-15}$
	0.9009	0.5697	0.3311	1.0536	$3.73 * 10^{-17}$
	0.5058	0.1577	0.3480	1.4061	$3.16 * 10^{-13}$

Table 4
Accuracy of $\mathcal{F}_{layernorm}$ protocol.

Protocol	input	secret shares		secret result	error
		share A	share B		
$\mathcal{F}_{layernorm}$	1.4548	0.8665	0.5883	1.4825	$2.5 * 10^{-17}$
	−0.8710	−0.5767	−0.2943	−0.8277	$5.5 * 10^{-17}$
	0.3091	−0.0368	0.3459	0.3444	$7.3 * 10^{-17}$
	−1.0435	−0.8375	−0.2061	−0.9992	$4.3 * 10^{-17}$

6.1. Accuracy of protocols

We analyzed the accuracy of the protocols used in SecureTLM under the assumption of data types being floating-point. Specifically, the execution results from each protocol in SecureTLM, which are first recovered, and then compared with the execution results from plain-text. As shown in Tables 1, 2, 3, 4, the error between the execution results of our protocols and plain-text was almost zero (errors apart from the mathematical approximations in the computation libraries), which indicates that our protocol is correct. This further reveals that our implemented protocols can accurately perform privacy inference tasks without modifying the model structure. Therefore, we do not need to retrain or fine-tune the model like existing methods to ensure its accuracy. Specifically, for the $\mathcal{F}_{Softmax}$ protocol, we conducted tests using a $4D$ vector and assuming 2-PC. The input values, secret shares, and plain-text results are presented in Table 1. It is evident that the error is nearly zero, with discrepancies typically appearing starting at 2^{-17}. For display purposes, our table includes results rounded to four decimal places. For $\mathcal{F}_{gelu}$ and $\mathcal{F}_{layernorm}$ we take the same setting as for $\mathcal{F}_{Softmax}$, and the results shown in Table 2 and 4 respectively. And for $\mathcal{F}_{mul}$, considering the seamless extension of $\mathcal{F}_{mul}$ to matrix multiplication, we conducted tests specifically targeting 2×2 matrix multiplication, the error still starts at 2^{-17}. We also evaluate the $\mathcal{F}_{invsquare}$ [40] SecureTLM used.

Table 5
Comparison the Mul efficiency of plain-text, OT-base-$\mathcal{F}_{mul}$, and $\mathcal{F}_{mul}$.

Methods	Scale	Comm(MB)	Rounds	Times(S)
Plain-text	$64 \times 128 \times 128$	0	0	0.002
	$64 \times 128 \times 256$	0	0	0.005
	$64 \times 256 \times 256$	0	0	0.011
	$128 \times 256 \times 256$	0	0	0.019
	$256 \times 512 \times 512$	0	0	0.218
$\mathcal{F}_{mul}$	$64 \times 128 \times 128$	32.0	2	0.119
	$64 \times 128 \times 256$	64.00	2	0.231
	$64 \times 256 \times 256$	128.00	8	0.281
	$128 \times 256 \times 256$	256.00	2	0.532
	$256 \times 512 \times 512$	512.00	2	2.628
OT-base-$\mathcal{F}_{mul}$	$64 \times 128 \times 128$	59.89	25	0.505
	$64 \times 128 \times 256$	116.82	25	0.948
	$64 \times 256 \times 256$	130.09	27	1.671
	$128 \times 256 \times 256$	461.56	31	2.642
	$256 \times 512 \times 512$	3722.72	39	20.101

Table 6
Comparison the Mul efficiency of plain-text, HE-base-$\mathcal{F}_{mul}$, and $\mathcal{F}_{mul}$.

Methods	Scale	Comm(MB)	Rounds	Times(S)
Plain-text	$8 \times 4096 \times 1$	0	0	< 0.001
	$16 \times 4096 \times 1$	0	0	< 0.001
	$16 \times 8192 \times 1$	0	0	< 0.001
	$32 \times 4096 \times 1$	0	0	< 0.001
	$32 \times 8192 \times 1$	0	0	< 0.001
$\mathcal{F}_{mul}$	$8 \times 4096 \times 1$	1.00	2	0.001
	$16 \times 4096 \times 1$	2.00	2	0.002
	$32 \times 4096 \times 1$	4.00	2	0.005
	$16 \times 8192 \times 1$	8.00	2	0.007
	$32 \times 8192 \times 1$	16.00	2	0.012
HE-base-$\mathcal{F}_{mul}$	$8 \times 4096 \times 1$	0.53	2	0.120
	$16 \times 4096 \times 1$	0.53	2	0.167
	$32 \times 4096 \times 1$	0.53	2	0.278
	$16 \times 8192 \times 1$	80.19	2	1.090
	$32 \times 8192 \times 1$	80.19	2	1.339

6.2. Performance of protocols

For each protocol in SecureTLM, we evaluated the communication overheads, number of communication rounds, and computational efficiency at different scales, assuming that the data types are fixed-point. Additionally, we tested the performance of end-to-end SecureTLM private inference. Concretely, to evaluate the computational efficiency, we assessed $\mathcal{F}_{mul}$ across specific scales and compared it with plain-text multiplication, OT-based and HE-based multiplication protocols (The HE-base-$\mathcal{F}_{mul}$ and OT-base-$\mathcal{F}_{mul}$ are implemented based SCI from EzPC library [45]. The results are obtained by averaging five computations.) First, we compare the $\mathcal{F}_{mul}$, OT-base-$\mathcal{F}_{mul}$ and plain-text in scenario where the matrix dimension are $64 \times 128 \times 128$, $64 \times 128 \times 256$, $64 \times 256 \times 128$, $128 \times 256 \times 256$ and $256 \times 512 \times 512$ respectively. As Table 5 shown, the results revealed that our multiplication protocol reduces communication rounds by an average of 15.7×, with a computational efficiency improvement of 4.24×, 4.10×, 5.95×, 5.06×, 7.64× respectively in different settings.

After that, we compare the $\mathcal{F}_{mul}$, HE-base-$\mathcal{F}_{mul}$ and plain-text in scenarios where the matrix dimension are $8 \times 4096 \times 1$, $16 \times 4096 \times 1$, $16 \times 8192 \times 1$, $32 \times 4096 \times 1$ and $32 \times 8192 \times 1$ respectively. As Table 6 shows, the results revealed that our multiplication protocol exhibited the computational efficiency improvement of 120×, 83.5×, 55.6×, 155.7×, 111.6× compared to the HE-based protocol in different settings.

To facilitate the observation of efficiency differences among $\mathcal{F}_{mul}$, OT-based, and HE-based multiplication, the bar chart has been plotted in this study, as shown in Fig. 4. The x-axis represents the data scale, while the y-axis represents the computation time (only the comparisons for the first four dimensions are presented, for ease of display, the precision of all protocols, except for the plain-text, was reduced by a factor of 10^{-1}).

Moreover, we performed tests on these non-linear protocols. Table 7 presents the runtime and communication details of the protocols. We can see that the runtime of these non-linear protocols is in the millisecond range. In particular, the runtime is all less than 0.0182 ms.

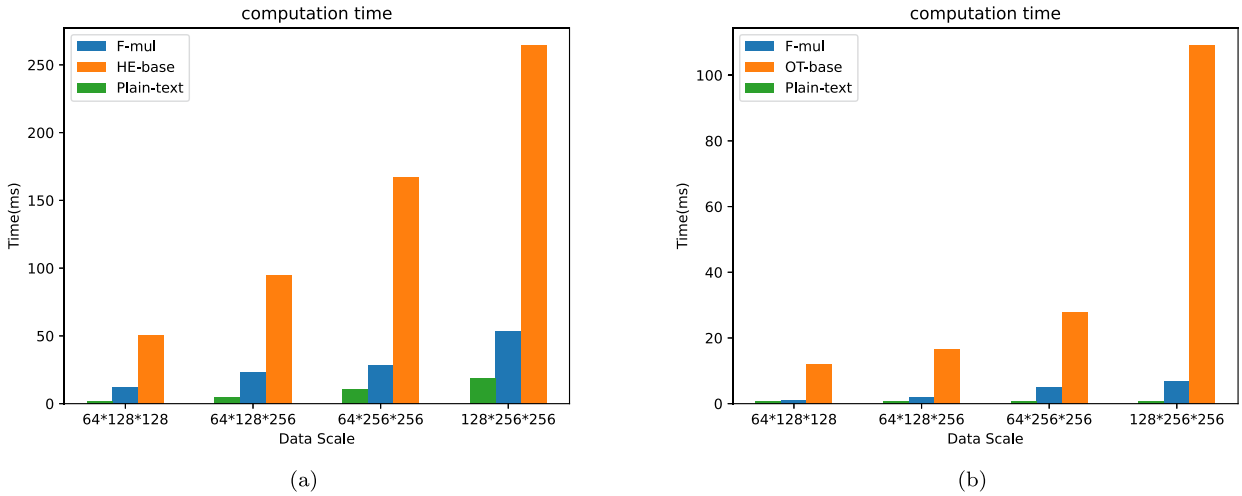

Fig. 4. The execution efficiency $\mathcal{F}_{mul}$, OT-base-$\mathcal{F}_{mul}$ and HE-base-$\mathcal{F}_{mul}$.

Table 7
Runtime and communication of non-linear protocols.

Length of input sequence	Comm(bytes)	Rounds	Times(ms)
$\mathcal{F}_{trans}$	892	1	0.0011
$\mathcal{F}_{Softmax}$	967	2	0.0016
$\mathcal{F}_{gelu}$	5657	6	0.0065
$\mathcal{F}_{inversquare}$ (iters = 10)	23511	31	0.0170
$\mathcal{F}_{layernorm}$	25575	33	0.0182

Table 8
Runtime and communication of SecureTLM private inference.

Length of input sequence	Times(s)	Comm(MB)
128	2.5982	7.2042
256	4.4846	14.0578
512	8.2379	27.7752
1024	15.9651	55.1931
2048	30.7136	89.0819
4096	60.6599	219.7239

6.3. Performance of SecureTLM

In SecureTLM private inference tasks, a parameters loading interface is provided. Therefore, in this experiment, for convenience, we substitute the parameters with randomly generated values of the same scale. The maximum length of input text sequence T does not exceed 4096. For comparison and practicality, we evaluated the runtime and communication overheads of SecureTLM for different input lengths. The runtime and communication overheads, which change with the length of the input sequence, can be found in Table 8. The number of heads is set to 12, and the model dimension is set to 16.

Furthermore, we also tested the end-to-end inference time and communication efficiency under different settings, including the length of the input sequence, the number of heads in the multi-head attention mechanism, and the dimensions of the model. Specifically, for different input sequence lengths, we set the model dimensions to 16 and the number of heads to 12; for different numbers of heads in the multi-head attention mechanism, we set the input length to 256 and the model dimensions to 16; for different model dimensions, we set the input length to 256 and the number of heads in the multi-head attention mechanism to 12. As shown in Fig. 5, the experiments demonstrated that with the increase in parameter size, both inference time and communication exhibit linear growth.

Through statistical analysis of the experimental results from 6.1, 6.2 and 6.3, we have validated the effectiveness of our design goals (3.3) in different scenarios. In terms of security, we have provided theoretical proofs. Additionally, we have demonstrated

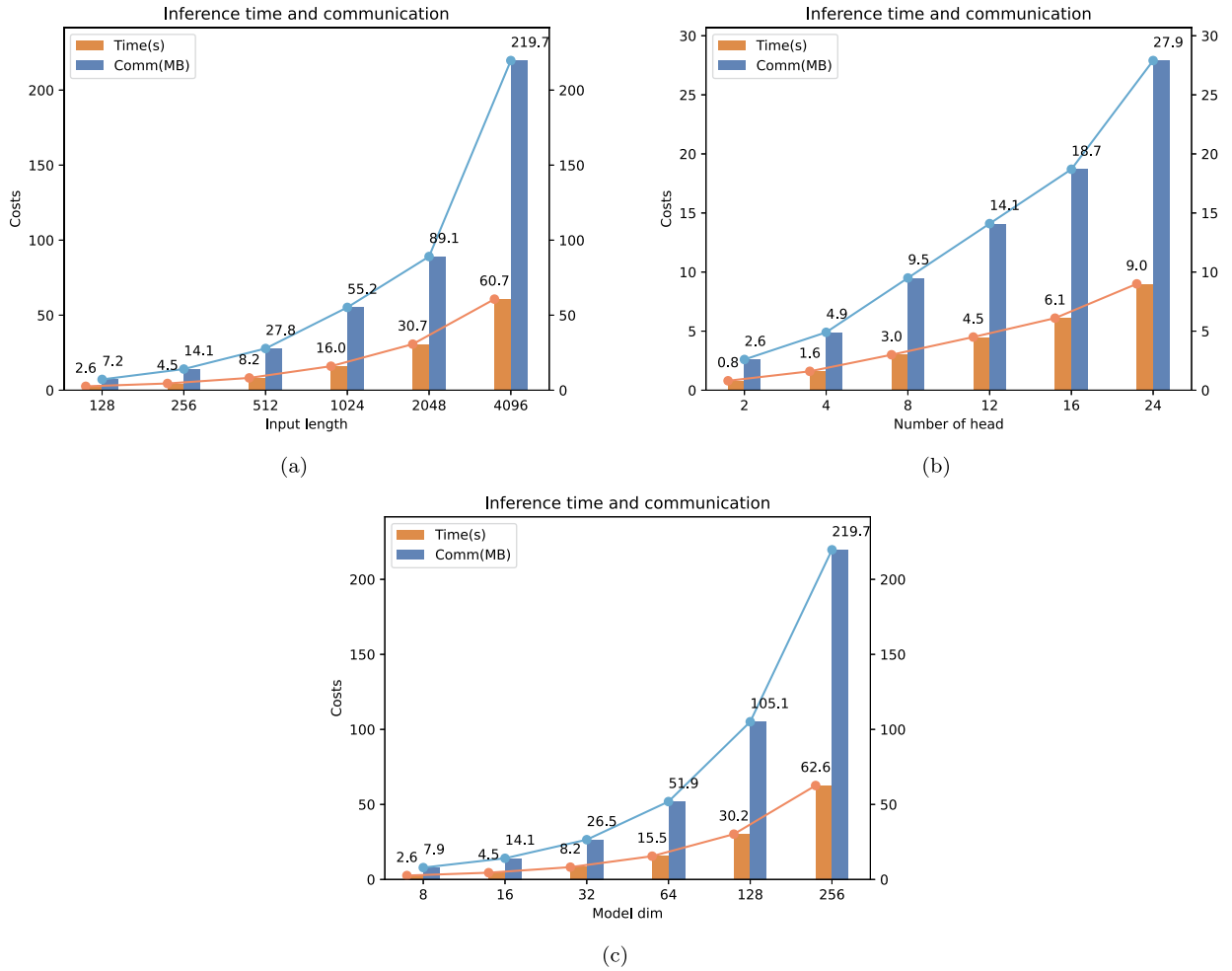

Fig. 5. The running time and communication under different settings.

the accuracy of the inference results by comparing them with the plain-text inference task. Additionally, we have compared the multiplication protocol with two difference approaches, and our protocols have shown a clear superiority in terms of efficiency.

7. Conclusion

In this work, we meticulously design a private inference framework for TLM using secure multi-party computation (MPC), safeguarding private data under the honest-but-curious model. Furthermore, the framework of SecureTLM adopts the "Dealer" setting. In this setting, we carefully design an efficient multiplication protocol, which only needs one round of interaction to complete the computation, and improves the execution efficiency by at least 4.10× and 55.6× compared with the OT-base and HE-base multiplication protocols respectively. In addition, we use the hybrid secret sharing technique to design accurate and non-replacement protocols for the nonlinear operators Softmax, GeLU, and LayerNorm, which do not need rude replacement model structures, so there is no need to retraining or fine-tuning the model. We believe that our work can advance the progress of private inference in AI large models and make it more secure.

CRediT authorship contribution statement

Yuntian Chen: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation. **Xianjia Meng:** Supervision. **Zhiying Shi:** Validation, Software, Data curation. **Zhiyuan Ning:** Validation. **Jingzhi Lin:** Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] T. Wolf, L. Debut, V. Sanh, et al., Transformers: state-of-the-art natural language processing, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2020, pp. 38–45.
[2] L. Zhang, S. Wang, F. Yuan, et al., Lifelong language learning with adaptive uncertainty regularization, Inf. Sci. 622 (2023) 794–807.
[3] I. Rivera Trigueros, Machine translation systems and quality assessment: a systematic review, Lang. Resour. Eval. 56 (2) (2022) 593–619.
[4] L. Zhang, J. Zhang, X. Ke, et al., A survey on complex factual question answering, AI Open 4 (2023) 1–12.
[5] H. Zhang, H. Song, S. Li, et al., A survey of controllable text generation using transformer-based pre-trained language models, ACM Comput. Surv. 56 (3) (2023) 1–37.
[6] K. Han, Y. Wang, H. Chen, et al., A survey on vision transformer, IEEE Trans. Pattern Anal. Mach. Intell. 45 (1) (2022) 87–110.
[7] T. Lin, Y. Wang, X. Liu, et al., A survey of transformers, AI Open (2022).
[8] D. Cheng, Y. Li, S. Xia, et al., A fast granular-ball-based density peaks clustering algorithm for large-scale data, IEEE Trans. Neural Netw. Learn. Syst. (2023).
[9] A. Radford, K. Narasimhan, T. Salimans, et al., Improving Language Understanding by Generative Pre-Training, 2018.
[10] J. Devlin, M. Chang, K. Lee, et al., Bert: pre-training of deep bidirectional transformers for language understanding, arXiv preprint arXiv:1810.04805, 2018.
[11] W.D. Heaven, Language models like gpt-3 could herald a new type of search engine, in: Ethics of Data and Analytics, Auerbach Publications, 2022, pp. 57–59.
[12] W. Fan, Z. Zhao, J. Li, et al., Recommender systems in the era of large language models (llms), arXiv preprint arXiv:2307.02046, 2023.
[13] M. Salvagno, F.S. Taccone, A.G. Gerli, et al., Can artificial intelligence help for scientific writing?, Crit. Care 27 (1) (2023) 1–5.
[14] L. Floridi, M. Chiriatti, Gpt-3: its nature, scope, limits, and consequences, Minds Mach. 30 (2020) 681–694.
[15] Y. Wu, L. Liu, C. Pu, et al., A comparative measurement study of deep learning as a service framework, IEEE Trans. Serv. Comput. 15 (1) (2019) 551–566.
[16] A. Boulemtafes, A. Derhab, Y. Challal, A review of privacy-preserving techniques for deep learning, Neurocomputing 384 (2020) 21–45.
[17] K. Cheng, N. Xi, X. Liu, et al., Private inference for deep neural networks: a secure, adaptive, and efficient realization, IEEE Trans. Comput. 72 (12) (2023) 3519–3531.
[18] J. Zhao, H. Zhu, F. Wang, et al., Efficient and privacy-preserving tree-based inference via additive homomorphic encryption, Inf. Sci. 650 (2023) 119480.
[19] A.C. Yao, Protocols for secure computations, in: 23rd Annual Symposium on Foundations of Computer Science (sfcs 1982), IEEE, 1982, pp. 160–164.
[20] A. Acar, H. Aksu, A.S. Uluagac, et al., A survey on homomorphic encryption schemes: theory and implementation, ACM Comput. Surv. 51 (4) (2018) 1–35.
[21] N. Kumar, M. Rathee, N. Chandran, et al., Cryptflow: secure tensorflow inference, in: 2020 IEEE Symposium on Security and Privacy (SP), IEEE, 2020, pp. 336–353.
[22] S. Tan, B. Knott, Y. Tian, et al., Cryptgpu: fast privacy-preserving machine learning on the gpu, in: 2021 IEEE Symposium on Security and Privacy (SP), IEEE, 2021, pp. 1021–1038.
[23] D. Kim, C. Guyot, Optimized privacy-preserving cnn inference with fully homomorphic encryption, IEEE Trans. Inf. Forensics Secur. 18 (2023) 2175–2187.
[24] M. Hao, H. Li, H. Chen, et al., Fastsecnet: an efficient cryptographic framework for private neural network inference, IEEE Trans. Inf. Forensics Secur. 18 (2023) 2569–2582.
[25] H. Zhang, F. Zhang, H. Wang, et al., A novel privacy-preserving graph convolutional network via secure matrix multiplication, Inf. Sci. 657 (2024) 119897.
[26] Q. Feng, D. He, Z. Liu, et al., Securenlp: a system for multi-party privacy-preserving natural language processing, IEEE Trans. Inf. Forensics Secur. 15 (2020) 3709–3721.
[27] M. Hao, H. Li, H. Chen, et al., Iron: private inference on transformers, Adv. Neural Inf. Process. Syst. 35 (2022) 15718–15731.
[28] X. Liu, Z. Liu, Llms can understand encrypted prompt: towards privacy-computing friendly transformers, arXiv preprint arXiv:2305.18396, 2023.
[29] Y. Akimoto, K. Fukuchi, Y. Akimoto, et al., Privformer: privacy-preserving transformer with mpc, in: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P), IEEE, 2023, pp. 392–410.
[30] T. Chen, H. Bao, S. Huang, et al., The-x: privacy-preserving transformer inference with homomorphic encryption, arXiv preprint arXiv:2206.00216, 2022.
[31] D. Li, R. Shao, H. Wang, et al., Mpcformer: fast, performant and private transformer inference with mpc, arXiv preprint arXiv:2211.01452, 2022.
[32] X. Hou, J. Liu, J. Li, et al., Ciphergpt: secure two-party gpt inference, in: Cryptology ePrint Archive, 2023.
[33] Q. Pang, J. Zhu, H. Möllering, et al., Bolt: privacy-preserving, accurate and efficient inference for transformers, in: Cryptology ePrint Archive, 2023.
[34] C. Juvekar, V. Vaikuntanathan, A. Chandrakasan, Gazelle: a low latency framework for secure neural network inference, in: 27th USENIX Security Symposium (USENIX Security 18), 2018, pp. 1651–1669.
[35] D. Beaver, Efficient multiparty protocols using circuit randomization, in: Advances in Cryptology—CRYPTO'91: Proceedings 11, Springer, 1992, pp. 420–432.
[36] M. Kelkar, P.H. Le, M. Raykova, et al., Secure Poisson regression, in: 31st USENIX Security Symposium (USENIX Security 22), USENIX Association, Boston, MA, 2022, pp. 791–808.
[37] Y. Lindell, How to simulate it–a tutorial on the simulation proof technique, in: Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich, 2017, pp. 277–346.
[38] W.j. Lu, Z.c. Huang, Z. Gu, et al., Bumblebee: secure two-party inference framework for large transformers, in: Cryptology ePrint Archive, 2023.
[39] G. He, Y. Ren, M. Bian, et al., Privacy-enhanced and non-interactive linear regression with dropout-resilience, Inf. Sci. 632 (2023) 69–86.
[40] S. Wagh, S. Tople, F. Benhamouda, et al., Falcon: honest-majority maliciously secure framework for private deep learning, arXiv preprint arXiv:2004.02229, 2020.
[41] W.j. Lu, Z.c. Huang, Q.z. Zhang, et al., Squirrel: a scalable secure two-party computation framework for training gradient boosting decision tree, in: Cryptology ePrint Archive, 2023.
[42] Y. Dong, X. Chen, X. Song, et al., Flexbnn: fast private binary neural network inference with flexible bit-width, IEEE Trans. Inf. Forensics Secur. (2023).
[43] Z. Xia, Q. Gu, W. Zhou, L. Xiong, J. Weng, N. Xiong, Str: secure computation on additive shares using the share-transform-reveal strategy, IEEE Trans. Comput. (2021).
[44] C. Gao, J. Yu, Securerc: a system for privacy-preserving relation classification using secure multi-party computation, Comput. Secur. 128 (2023) 103142.
[45] D. Rathee, M. Rathee, R.K.K. Goli, et al., Sirnn: a math library for secure rnn inference, in: 2021 IEEE Symposium on Security and Privacy (SP), IEEE, 2021, pp. 1003–1020.